University of Kragujevac



Artificial intelligence

Project on the subject of virtual reality systems

Development of a memory game

Student:

Babaev Bogdan 0866511

Kragujevac, June 2024

Contents

Introduction 3

Description of the project 3

1. Preparation for development 4

2. Program development 5

3. Main part of the program. 7

4. Testing the program 8

List of used literature 11

Introduction

This project shows the development of a Memory game. The goal of the game is to open two cards at a time to find all pairs with the same pictures). The game is made in unity for a VR helmet, but can also be run on a simple computer.

Description of the project

The project is an augmented reality memory game developed on the Unity platform for virtual reality helmets or emulators. The playing field is 6x6 in size, where each pair of cards has the same image. The goal of the game is to open two cards at a time to find all pairs with the same pictures. The game can be run on a weak PC, which makes it accessible to a wide range of users.

1. Preparation for development

Before the start of development, preparations were made, a project was created in unity, and a project folder was also created, in which the subfolders are located

1) Assets – The folder where all the project files are stored
2) Sprites - The folder for storing images for the project
3) Scripts - a folder containing scripts for managing user actions

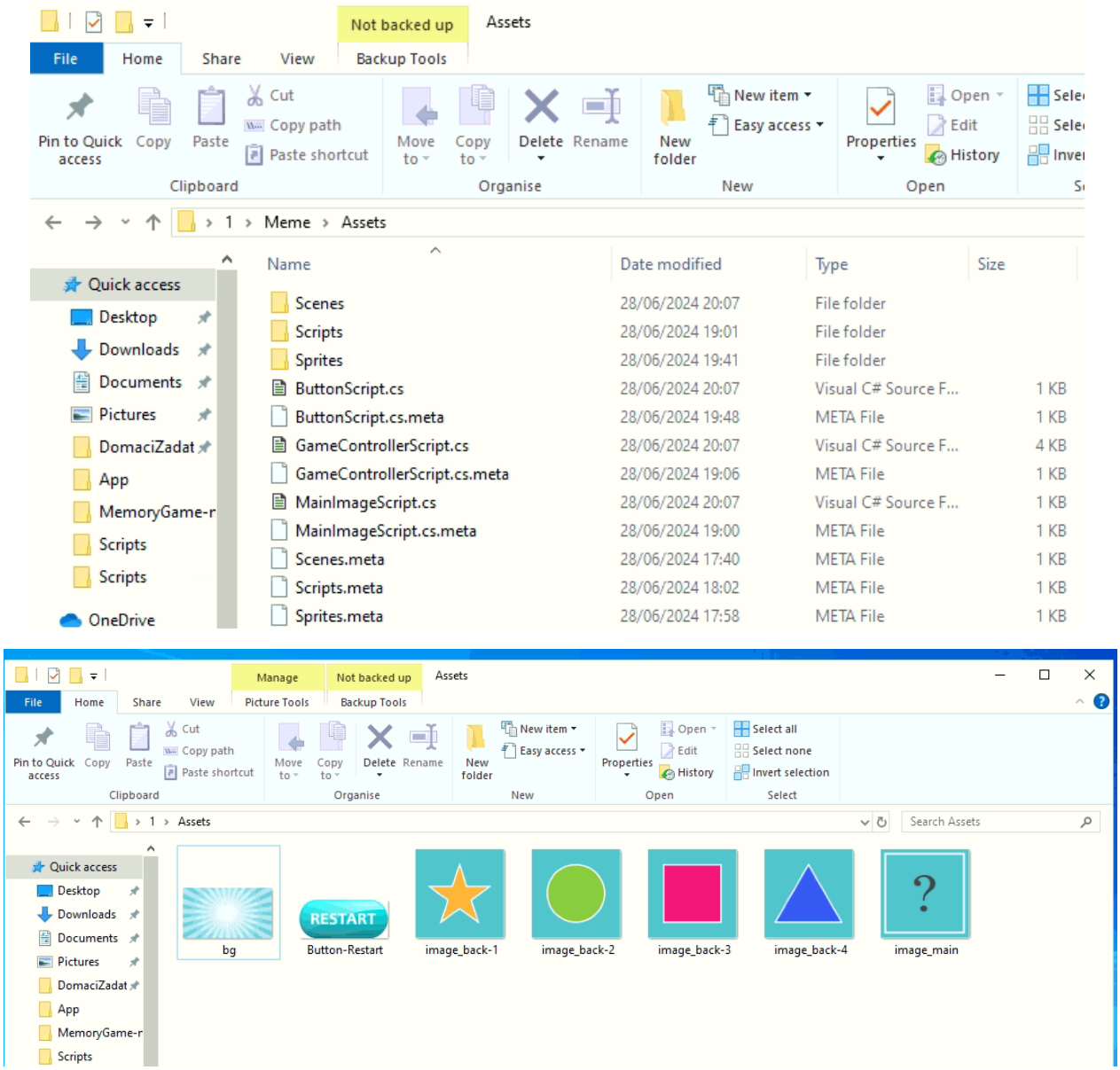


Figure 1: folder with files for the project

2. Program development

The program code for Unity is developed in C#. The project was developed in unity under the Windows operating system.

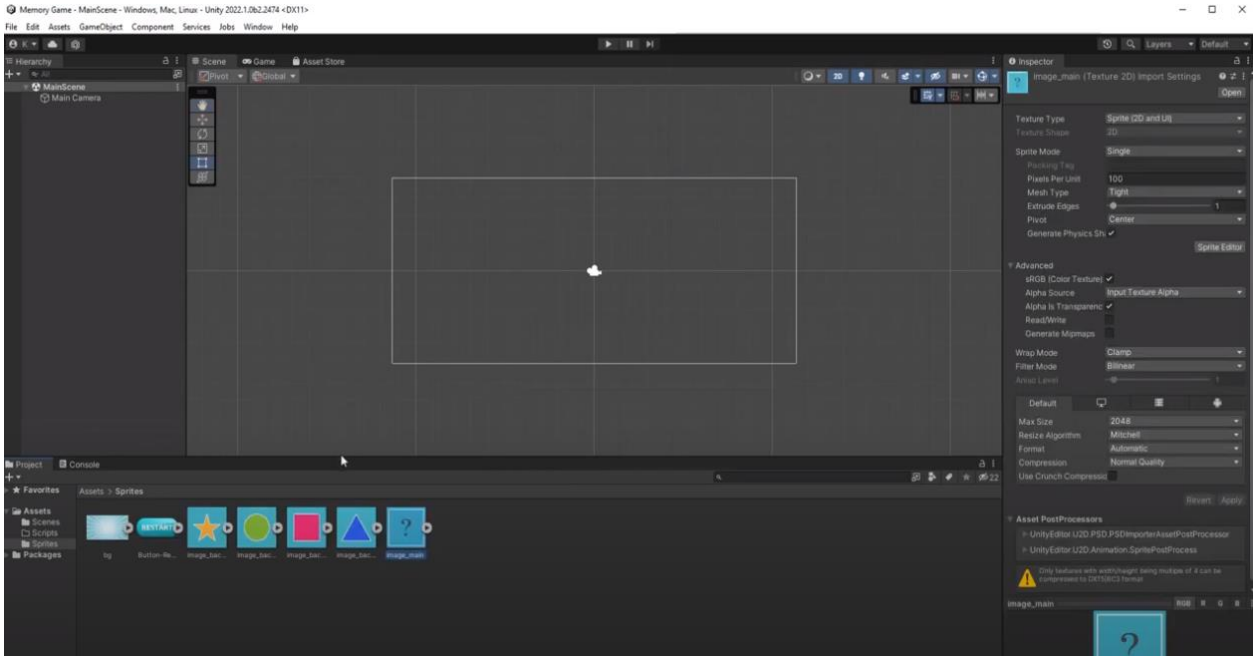


Figure 2: The unity project

The structure of file storage in the project

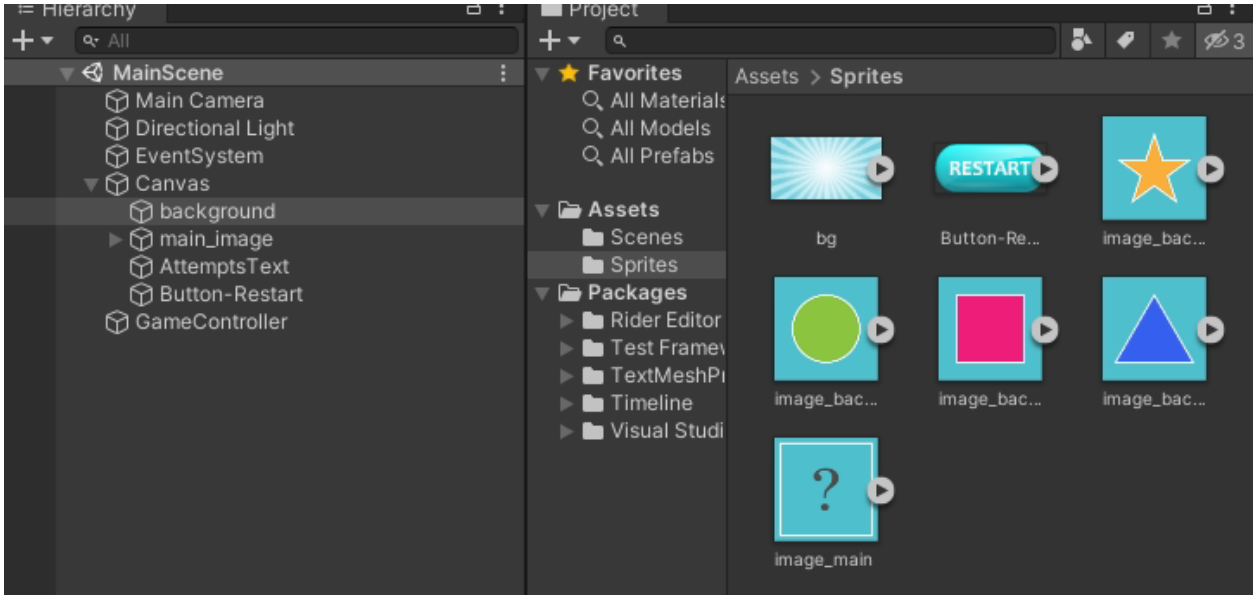


Figure 3: File structure

A card with a question mark is used as a game object, on the back of which another picture is randomly displayed, and a script is also attached to the picture, which allows you to flip the picture when clicked and shows another one.

The program code, which will be analyzed below, shows how other cards are created by analogy with this one.

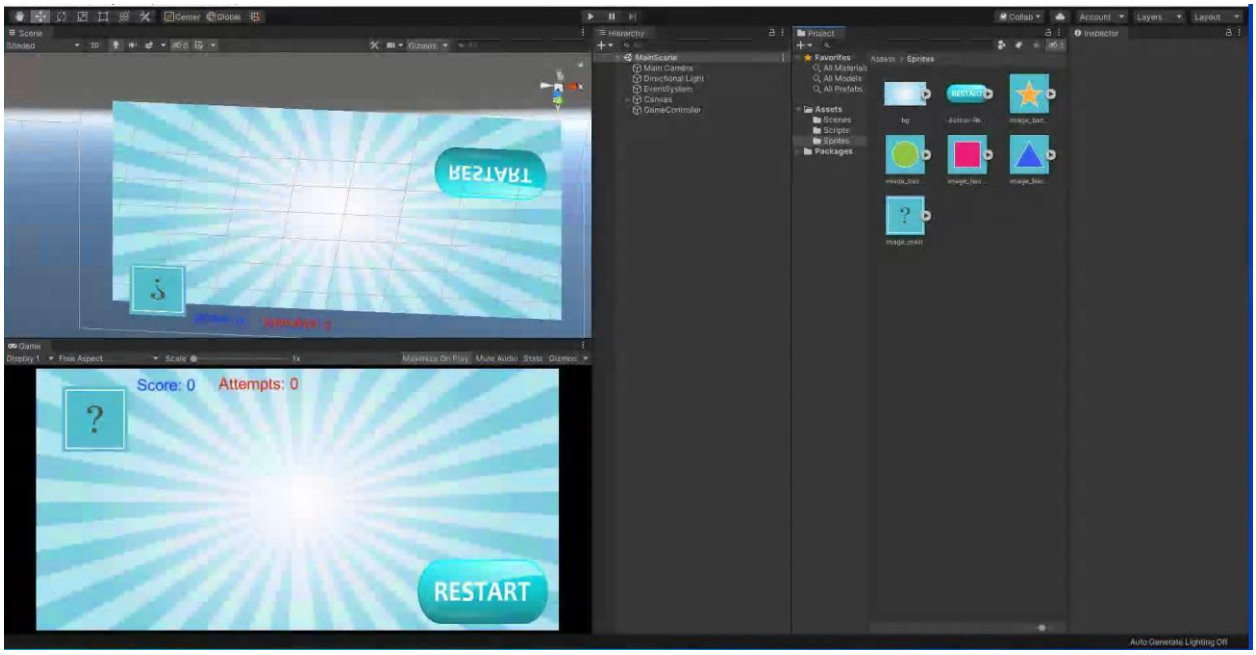


Figure 4: the displayed interface of the game

The project has been created, let's move on to developing script files in C# that are linked to the game buttons, text and cards.

3. Main part of the program.

There are three main scripts in the program:

1) MainImageScript – it stores the logic of card flip when you click on the card
2) ButtonScript – the button script
3) GameControllerScript – the logic of the game as a whole

The first step was to create MainImageScript in unity and open it in the visual studio. This file is attached to the double pictures, which are turned over and thus perform the main function in the game.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MainImageScript : MonoBehaviour
{
    [SerializeField] private GameObject image_unknown;
    [SerializeField] private GameControllerScript gameController;

    public void OnMouseDown()
    {
        if (image_unknown.activeSelf && gameController.canOpen)
        {
            image_unknown.SetActive(false);
            gameController.imageOpened(this);
        }
    }

    private int _spriteId;
    public int spriteId
    {
        get { return _spriteId; }
    }

    public void ChangeSprite(int id,Sprite image)
    {
        _spriteId = id;
        GetComponent<SpriteRenderer>().sprite = image; //Gets the
    }

    public void Close()
    {
        image_unknown.SetActive(true); // Hide image
    }
}
```

Variables:

image_unknown is an object representing a face-down card.

gameController - link to the game controller.

OnMouseDown method:

When you click on the card, it is checked whether it is closed and whether it can be opened.

If the card is closed and can be opened, it is revealed (the image is hidden) and the game controller is notified that it has been opened.

SpriteId property:

Stores the identifier of the sprite (image) of the map.

ChangeSprite method:

Changes the sprite (image) of the map and sets its identifier.

Close method: Closes the card (shows an image of the closed c).

Figure 5: The MainImageScript

Thus, this script controls the behavior of cards in the memory game: opening, closing and changing the image of cards.

The second file is a Button Script, which stores the logic of the restart button to start a new game.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ButtonScript : MonoBehaviour
{
    [SerializeField] private GameControllerScript gameController;
    [SerializeField] private string functionOnClick;


    public void OnMouseOver()
    {
        SpriteRenderer sprite = GetComponent<SpriteRenderer>();
        if(sprite != null)
        {
            sprite.color = Color.cyan;
        }
    }

    public void OnMouseDown()
    {
        transform.localScale = new Vector3(0.3f, 0.3f, 1.0f);
    }

    public void OnMouseUp()
    {
        transform.localScale = new Vector3(0.2f, 0.2f, 1.0f);
        if(gameController != null)
        {
            gameController.SendMessage(functionOnClick);
        }
    }

    public void OnMouseExit()
    {
        SpriteRenderer sprite = GetComponent<SpriteRenderer>();
        if (sprite != null)
        {
            sprite.color = Color.white;
        }
    }
}
```

Variables:

gameController - link to the game controller.

functionOnClick - the name of the function to be called when the button is clicked.

OnMouseOver method:

When you hover the mouse over a button, if it has a SpriteRenderer component, the color of the button changes to blue.

OnMouseDown method:

When you press the button, its size decreases.

OnMouseUp method:

When the button is released, its size is restored.

If the game controller is not null, the controller function whose name is specified in functionOnClick is called.

OnMouseExit method:

When the mouse cursor leaves the button, its color changes back to white (if the button has a SpriteRenderer component).

Figure 6: The ButtonScript

Thus, this script manages the visual effects when interacting with the button and calling a function in the game controller when the button is clicked.

And the last GameControllerScript file, it stores the logic of the game, it stores the logic of the game for memorization, initialization of the playing field, processing of card opening events and scoring and the number of attempts and restarting the game.

Constants:

columns and rows determine the size of the playing field (6x6).

Xspace and Yspace define the distance between maps along the X and Y axes.

Variables:

startObject - reference to the starting object of the map.

images - an array of sprites (images) for maps.

firstOpen and secondOpen - links to two open cards.

score - number of pairs found.

attempts - number of attempts.

scoreText and attemptsText - text fields to display the score and number of attempts.

Randomiser method

Shuffles the array with card identifiers.

Start method:

Initializes the playing field, creating and placing cards based on a shuffled array of IDs.

canOpen property:

Returns true if another card can be opened (that is, if secondOpen is null).

imageOpened method:

Called when a card is opened.

If this is the first card opened, it is stored in firstOpen.

If this is the second open card, it is saved in secondOpen and the CheckGuessed coroutine is launched.

Coroutine **CheckGuessed**:

Checks if the IDs of two open cards are the same.

If they match, the score increases.

If they do not match, closes both cards after a pause.

Increases the number of attempts and updates text fields.

Restart method: Reloads the scene to start the game over.

Let's run the emulator and see how the game works. After that, we will draw conclusions about the development of such a memorization game.

4. Testing the program

We launched the program, a playing field appeared in front of us, on which 36 cards are displayed, when clicked on, they turn over and show us the reverse side, in case of simultaneous opening of two identical cards, they remain open.

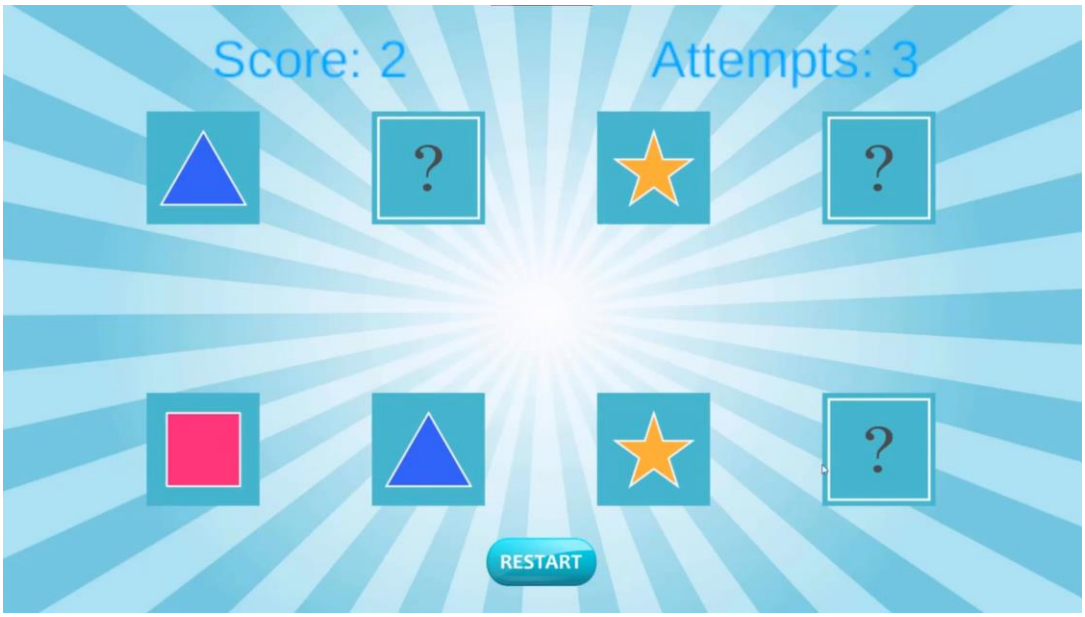


Figure 7: Testing the game

The development of the Memory game encountered technical challenges that prevented the full realization of the intended 6x6 grid for VR gameplay. Instead, the project successfully implemented a scaled-down version featuring a 4x4 grid playable on standard computers. This adaptation ensured broader accessibility while maintaining core gameplay mechanics centered on memory and image matching.

Moving forward, addressing technical barriers could enable future iterations to achieve the initial vision of a fully immersive VR experience.

List of used literature

1. Unity Documentation. Retrieved from Unity Official Website
2. C# Programming Guide. Retrieved from Microsoft Docs
3. Hololens: Install the tools. Retrieved from [Hololens: Install the toolsURL adresa](URL adresa)
4. Hololens: 1st Gen Hardware. Retrieved from [Hololens: 1st Gen HardwareURL adresa](URL adresa)
5. Hololens: Share screen demo. Retrieved from [Hololens: Share screen demoURL adresa](URL adresa)
6. Oculus Quest: Quest 2. Retrieved from [Oculus Quest: Quest 2URL adresa](URL adresa)
7. Oculus Quest: Help Center. Retrieved from [Oculus Quest: Help CenterURL adresa](URL adresa)
8. Oculus Quest: Quick Start Tutorials. Retrieved from [Oculus Quest: Quick Start TutorialsURL adresa](URL adresa)
9. Oculus Quest: Hand tracking - Unity tutorial. Retrieved from [Oculus Quest: Hand tracking - Unity tutorialURL adresa](URL adresa)
10. Magic Leap: Specifications. Retrieved from [Magic Leap: SpecificationsURL adresa](URL adresa)
11. Magic Leap: Magic Leap 2 Overview. Retrieved from [Magic Leap: Magic Leap 2 OverviewURL adresa](URL adresa)
12. Magic Leap: AR for Business. Retrieved from [Magic Leap: AR for BusinessURL adresa](URL adresa)